GUIA DE ESTUDO

Desafio Backend Go Processamento Distribuído de Apostas

O que foi construído em cada spec, por que foi feito assim e quais conceitos de Go aparecem, para quem vem de C#, Python e TypeScript.

Repositório: github.com/rafaelcorreadev1/junglegaming
Atualizado até a spec 02 · 13/10/2026

Como usar este guia

Cada spec concluída e testada vira um capítulo. Todos os capítulos seguem a mesma ordem:

1. **O que a spec pedia** — o requisito do desafio.
2. **O que foi construído** — os arquivos e o papel de cada um.
3. **Como funciona** — o fluxo, passo a passo.
4. **Conceitos de Go** — o que é novo, comparado com C# e TypeScript.
5. **Decisões** — as escolhas feitas e o motivo.
6. **Como foi verificado** — o que foi executado e o resultado.
7. **Perguntas de revisão** — para testar se você entendeu.

Leia com o código aberto ao lado. Os trechos aqui são recortes; o arquivo completo está no caminho indicado.

Progresso

#	SPEC	STATUS
01	Bootstrap: módulo Go, Fx, config, Docker Compose	Concluída
02	Money	Concluída
03	Domínio: Wallet, Ledger, WagerTransaction	A fazer
04	Schema e migrations	A fazer
05	Persistência e transação SQL	A fazer
06	Carteira: abertura, leitura, ledger paginado	A fazer
07	Operação: BET / WIN / LOSS e idempotência	A fazer
08	Reversões e referências pendentes	A fazer
09	Autenticação e autorização	A fazer
10	Outbox e eventos	A fazer
11	Consumidor SQS, inbox e DLQ	A fazer
12	Reconciliação	A fazer
13	Observabilidade	A fazer
14	Ciclo de vida Fx e shutdown	A fazer
15	Testes de integração, concorrência e recuperação	A fazer
16	Documentação e entrega	A fazer

O desafio em um parágrafo

Um serviço que movimenta carteiras de jogadores a partir de operações enviadas por provedores de jogos: aposta (`BET`), ganho (`WIN`), perda (`LOSS`), devolução (`REFUND`) e estorno (`ROLLBACK`). As operações chegam por HTTP e por fila (SQS), e o resultado financeiro precisa ser o mesmo nos dois caminhos, mesmo com várias instâncias rodando e com falhas no meio do processamento.

Mapa mental: Go para quem vem de C# e TypeScript

EM GO	O EQUIVALENTE QUE VOCÊ CONHECE
`go.mod` / `go.sum`	`.csproj` / `package.json` + lockfile
Pasta `internal/`	`internal` do C#: o compilador impede import de fora do módulo
Nome com maiúscula	`public` ; minúscula é privado ao pacote
`struct` + função `NewX`	Classe + construtor
`interface` implícita	Interface estrutural do TypeScript: não existe `implements`
`error` como retorno	Exceção, só que retornada e checada com `if err != nil`
`context.Context`	`CancellationToken`
`defer`	`finally` / `using`
Goroutine (`go f()`)	`Task.Run`
Uber Fx	`IServiceCollection` / modules do NestJS

SPEC 01

Bootstrap: módulo Go, Fx, config e Docker Compose

Um processo Go que sobe, valida a configuração, conecta no Postgres e responde dois endpoints de saúde. Ainda não há nenhuma regra de negócio.

1. O que a spec pedia

- Versão do Go declarada em `go.mod` e no `Dockerfile`; dependências versionadas.
- Composição da aplicação com Uber Fx, com injeção por construtores.
- Configuração validada na inicialização: o processo não sobe com config inválida.
- Ambiente local com Docker Compose e um `.env.example` sem segredos.

2. O que foi construído

ARQUIVO	PAPEL
`cmd/app/main.go`	Ponto de entrada: carrega a config e entrega o controle ao Fx
`internal/config/config.go`	Lê e valida as variáveis de ambiente
`internal/app/app.go`	A composição: lista os módulos que formam a aplicação
`internal/infra/observability/logger.go`	Logger em JSON
`internal/infra/postgres/pool.go`	Pool de conexões com o Postgres
`internal/infra/httpapi/server.go`	Rotas e servidor HTTP
`internal/infra/httpapi/health.go`	Endpoints `/health/live` e `/health/ready`
`Dockerfile`, `docker-compose.yml`	Imagem da aplicação e ambiente local

3. Como funciona

1. `main` lê as variáveis de ambiente e monta a `Config`.
2. Config inválida: imprime o erro e sai com código 1. Nada mais é criado.
3. O Fx recebe os construtores e descobre a ordem: logger, pool, handler, router, servidor.
4. O Fx executa os `OnStart`: ping no banco e depois abertura da porta HTTP.
5. O processo fica parado esperando um sinal (`SIGTERM` ou Ctrl+C).
6. No sinal, o Fx executa os `OnStop` na ordem inversa: o servidor HTTP para e depois o pool fecha.

4. Conceitos de Go

Erro é valor de retorno

Go não tem exceção. A função que pode falhar devolve o erro como último retorno e quem chama é obrigado a decidir o que fazer com ele.

```
cfg, err := config.Load()
if err != nil {
    fmt.Fprintln(os.Stderr, err)
    os.Exit(1)
}
```

Embrulhar erro com `%w`

O erro original fica guardado dentro do novo, como a `InnerException` do C#. É o que permite, mais à frente, classificar erros com `errors.Is`, uma exigência do desafio.

```
if err := pool.Ping(ctx); err != nil {
    return fmt.Errorf("ping postgres: %w", err)
}
```

Visibilidade pela primeira letra

`Config`, `Load` e `NewPool` começam com maiúscula e são públicos. `valueOr` e `writeJSON` começam com minúscula e só existem dentro do próprio pacote.

Struct, construtor e método

Não há classes. Os dados ficam em uma `struct`, o construtor é uma função comum chamada `NewAlgo` por convenção, e o método é uma função com um *receiver*, que faz o papel do `this`.

```
type HealthHandler struct {
    pool   *pgxpool.Pool
    logger *slog.Logger
}

func NewHealthHandler(pool *pgxpool.Pool, logger *slog.Logger) *HealthHandler {
    return &HealthHandler{pool: pool, logger: logger}
}

func (h *HealthHandler) Live(w http.ResponseWriter, _ *http.Request) {
    writeJSON(w, http.StatusOK, map[string]string{"status": "ok"})
}
```

O `*` indica ponteiro (referência ao mesmo objeto) e o `&` cria esse ponteiro. O `_` significa "recebo o parâmetro, mas não uso".

`context.Context` e `defer`

O `context` vai sempre como primeiro parâmetro e carrega cancelamento e prazo. No readiness, ele limita o ping a 2 segundos. O `defer` agenda a linha para o fim da função, com ou sem erro.

```
ctx, cancel := context.WithTimeout(r.Context(), readinessTimeout)
defer cancel()

if err := h.pool.Ping(ctx); err != nil { ... }
```

Goroutine

`go func() { ... }()` inicia uma execução concorrente. O servidor HTTP roda em uma goroutine porque `Serve` bloqueia até o servidor fechar, e a inicialização precisa continuar.

5. Uber Fx

O Fx é o container de injeção de dependência. Você não registra "interface para implementação": entrega as funções construtoras e ele lê as assinaturas.

```
func NewRouter(health *HealthHandler) http.Handler
```

O Fx lê isso como: para criar um `http.Handler`, preciso de um `*HealthHandler`. Ele procura quem produz esse tipo e chama na ordem certa.

FUNÇÃO	O QUE FAZ
`fx.Provide(NewX)`	Registra um construtor. Só executa se alguém precisar do resultado.
`fx.Invoke(f)`	Executa sempre. É o que força a construção do servidor.
`fx.Supply(valor)`	Registra um valor pronto, sem construtor. Usado para a `Config`.
`fx.Module("nome", ...)`	Agrupa os registros de um pacote.
`fx.Lifecycle`	Onde cada componente registra `OnStart` e `OnStop`.

A composição inteira, em `internal/app/app.go`:

```
func Options(cfg config.Config) fx.Option {
    return fx.Options(
        fx.Supply(cfg),
        fx.StartTimeout(cfg.ShutdownTimeout),
        fx.StopTimeout(cfg.ShutdownTimeout),
        fx.WithLogger(...),
        observability.Module,
        postgres.Module,
        httpapi.Module,
    )
}
```

E o ciclo de vida do pool, em `internal/infra/postgres/pool.go`:

```
lc.Append(fx.Hook{
    OnStart: func(ctx context.Context) error {
        if err := pool.Ping(ctx); err != nil {
            return fmt.Errorf("ping postgres: %w", err)
        }
        return nil
    },
    OnStop: func(context.Context) error {
        pool.Close()
        return nil
    },
})
```

Por que a ordem de parada sai certa sozinha. O servidor depende do handler, que depende do pool. Então o pool é construído primeiro e registra seu hook primeiro. O Fx executa os `OnStop` na ordem inversa dos `OnStart`: o servidor para de receber requisições antes de o pool fechar.

6. Decisões

DECISÃO	MOTIVO
A config devolve todos os erros de uma vez (`errors.Join`)	Quem configura o ambiente corrige tudo em uma tentativa
A config recebe uma função `getenv` em vez de chamar `os.Getenv`	O teste passa um mapa e não altera o ambiente real
A config é carregada no `main`, antes do Fx	Os timeouts do próprio Fx vêm dela
A porta é aberta no `OnStart`, fora da goroutine	Porta ocupada vira falha de inicialização, em vez de erro perdido em background
O ping no banco fica no `OnStart`	Criar o pool não abre conexão; só o ping valida o banco
Composição em `internal/app`, fora do `main`	Permite testar o grafo de dependências sem subir nada
Dockerfile em dois estágios, com imagem final *distroless*	Go gera um binário único; a imagem final não precisa de compilador nem de shell

Liveness e readiness

ENDPOINT	PERGUNTA QUE RESPONDE	VERIFICA
`GET /health/live`	O processo está de pé?	Nada externo
`GET /health/ready`	Consigo atender requisições?	Postgres (o SQS entra na spec 10)

Se o banco cair, o orquestrador para de mandar tráfego (ready falha), mas não reinicia o processo (live continua ok), porque reiniciar não resolveria.

7. Testes

O arquivo de teste fica ao lado do código, com sufixo `_test.go`, e usa só o pacote `testing` da biblioteca padrão. O formato idiomático é o teste em tabela, equivalente ao `[Theory]` do xUnit:

```
tests := []struct {
    name    string
    env     map[string]string
    wantErr string
}{
    {name: "missing database url", env: map[string]string{}, wantErr: "DATABASE_URL is
required"},
    ...
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        _, err := FromEnv(envFrom(tt.env))
        ...
    })
}
```

`internal/app/app_test.go` usa `fx.ValidateApp`: confere que todo construtor tem as dependências de que precisa, sem iniciar nada.

8. Como foi verificado

VERIFICAÇÃO	RESULTADO
`gofmt -l .` e `go vet ./...`	Sem saída
`go test ./...`	Passou
`go test -race ./...` (em container Linux)	Passou
`docker compose up --build`	`/health/live` e `/health/ready` respondem 200
Postgres parado	`/health/ready` 503, `/health/live` 200
Sem `DATABASE_URL` e com `LOG_LEVEL` inválido	Os dois erros impressos, código de saída 1
`docker compose stop app`	Servidor para, pool fecha, código de saída 0

> **O `-race` não roda direto no Windows.** O detector de condição de corrida exige um compilador C, que não vem com o Go no Windows. A saída é executar em container:
>
> `docker run --rm -v "${PWD}:/src" -v wagering-gomod:/go/pkg/mod -w /src golang:1.27 go test -race ./...`

9. Comandos do dia a dia

COMANDO	PARA QUÊ
`go run ./cmd/app`	Roda a aplicação (precisa de `DATABASE_URL`)
`go test ./...`	Roda todos os testes; `./...` é "este diretório e todos abaixo"
`go vet ./...`	Análise estática
`gofmt -l .`	Lista arquivos fora do padrão de formatação
`go mod tidy`	Ajusta `go.mod` e `go.sum` ao que o código importa
`docker compose up --build`	Sobe Postgres e aplicação

10. Perguntas de revisão

Por que existe um `fx.Invoke(func(*http.Server) {})` com corpo vazio?

Porque o Fx só constrói o que alguém pede. Sem essa linha ninguém pediria o servidor, e ele nunca seria criado nem iniciado.

O que acontece se a porta 8080 já estiver em uso?

O `net.Listen` falha dentro do `OnStart`, o Fx aborta a inicialização e executa os `OnStop` do que já tinha iniciado (o pool fecha).

Por que o pool é fechado depois do servidor HTTP no shutdown?

Porque o servidor ainda pode estar terminando requisições que usam o banco. Fechar o pool antes quebraria essas requisições.

Qual a diferença entre `fx.Provide` e `fx.Supply`?

`Provide` registra uma função que cria o valor; `Supply` registra o valor já pronto.

Por que `FromEnv` recebe uma função em vez de ler o ambiente diretamente?

Para o teste controlar as variáveis sem alterar o ambiente do processo, o que também permite rodar testes em paralelo.

Com o banco fora do ar, por que `/health/live` continua 200?

Liveness diz se o processo precisa ser reiniciado. Banco fora não se resolve reiniciando a aplicação; quem sinaliza isso é o readiness.

Ordem de leitura do código

1. `cmd/app/main.go`
2. `internal/config/config.go` e o teste ao lado
3. `internal/app/app.go`
4. `internal/infra/postgres/pool.go`
5. `internal/infra/httpapi/server.go` e `health.go`

SPEC 02

Money

O tipo que representa dinheiro em todo o sistema: centavos em `int64` mais a moeda. Nenhum valor monetário passa por ponto flutuante.

1. O que a spec pedia

- Value object imutável com valor e moeda.
- Criação a partir de string decimal, zero por moeda, soma, subtração, negação, comparação e serialização.
- Proibido `float32` e `float64` em parsing, cálculo, serialização e persistência. É critério eliminatório.
- Escala fixa de duas casas e moeda ISO 4217.
- Rejeitar vazio, `NaN`, `Infinity`, notação científica e casas a mais, sem arredondar.
- Operações só entre valores da mesma moeda.
- Tratar overflow no parsing, na soma, na subtração e na negação.

2. Por que não usar float

Ponto flutuante guarda números em base 2, e a maioria das frações decimais não tem representação exata nela. Em qualquer linguagem, `0.1 + 0.2` resulta em `0.30000000000000004`. Em um sistema financeiro, esse resíduo vira centavo sobrando ou faltando depois de milhares de operações.

A saída é guardar a quantia como inteiro na menor unidade da moeda: `25.00` vira `2500` centavos. Inteiros somam e subtraem de forma exata.

3. O que foi construído

ARQUIVO	PAPEL
`internal/domain/money.go`	O tipo `Money`, seus erros e o parser
`internal/domain/money_test.go`	Testes em tabela; 100% das instruções cobertas

O pacote `domain` importa só a biblioteca padrão. Essa é a regra do desafio: o domínio não conhece Fx, HTTP, SQS nem banco.

A interface pública

FUNÇÃO OU MÉTODO	O QUE FAZ
`ParseMoney("25.00", "BRL")`	Cria a partir do contrato externo (string decimal)
`MoneyFromMinorUnits(2500, "BRL")`	Cria a partir de centavos; é o caminho de volta do banco
`ZeroMoney("BRL")`	Zero na moeda informada
`Add` , `Sub` , `Neg`	Devolvem um novo `Money` e um erro
`Cmp` , `Equal`	Comparação; `Cmp` devolve -1, 0 ou 1
`IsZero` , `IsPositive` , `IsNegative`	Perguntas que as regras de negócio vão usar
`Amount()` , `MinorUnits()` , `Currency()`	Leitura: `"25.00"` , `2500` , `"BRL"`
`MarshalJSON` , `UnmarshalJSON`	Conversão de e para `{"amount":"25.00","currency":"BRL"}`

4. Como funciona

Imutabilidade

```
type Money struct {
    minorUnits int64
    currency   string
}
```

Os dois campos começam com minúscula, então nenhum outro pacote consegue alterá-los. Os métodos usam receiver por valor (`func (m Money)` , sem `*`): cada chamada trabalha em uma cópia, e o resultado é sempre um `Money` novo. É o equivalente a um `readonly record struct` do C#.

O parser, passo a passo

1. Separa o sinal `-` , se houver. O sinal `+` não é aceito.
2. Divide no ponto: parte inteira e parte decimal.
3. Exige só dígitos de `0` a `9` nas duas partes. Isso elimina `NaN` , `Infinity` , `1e3` , espaços e vírgulas de uma vez.
4. Rejeita mais de duas casas decimais. Não há arredondamento.
5. Completa a parte decimal com zeros: `"25.5"` vira `"2550"` .
6. Converte dígito a dígito para `int64` , checando overflow antes de cada multiplicação.

Não se usa `strconv.ParseFloat` nem nenhuma função de conversão pronta: o parser é uma validação de caracteres seguida de aritmética de inteiros.

Overflow

Em Go, somar além do limite de um `int64` não gera erro: o número "dá a volta" e vira negativo, em silêncio. Por isso a checagem acontece **antes** da operação:

```
a, b := m.minorUnits, other.minorUnits
if (b > 0 && a > math.MaxInt64-b) || (b < 0 && a < math.MinInt64-b) {
    return Money{}, fmt.Errorf("%w: %s + %s", ErrMoneyOverflow, m, other)
}
return Money{minorUnits: a + b, currency: m.currency}, nil
```

A leitura é: "se `b` é positivo e `a` já está a menos de `b` do teto, a soma estouraria". A comparação é reescrita de forma que ela própria nunca estoure.

> **O caso da negação.** Um `int64` vai de -9.223.372.036.854.775.808 a +9.223.372.036.854.775.807: existe um negativo a mais. Negar o menor valor não tem resultado possível, então `Neg()` devolve erro nesse caso.

Erros classificáveis

```
var (
    ErrInvalidMoney     = errors.New("money is not initialized")
    ErrInvalidAmount    = errors.New("invalid money amount")
    ErrInvalidCurrency  = errors.New("invalid currency")
    ErrCurrencyMismatch = errors.New("currency mismatch")
    ErrMoneyOverflow    = errors.New("money amount overflow")
)
```

São os *erros sentinela*: variáveis públicas que servem de identidade do erro. Quem recebe o erro pergunta qual é com `errors.Is`, mesmo que ele tenha sido embrulhado com detalhes pelo `%w`:

```
_, err := brl.Add(usd)
if errors.Is(err, domain.ErrCurrencyMismatch) { ... }
```

Em C# isso seria um `catch (CurrencyMismatchException)`. Mais à frente, a camada HTTP usa essa classificação para decidir entre 400, 409 e 422.

O valor zero de uma struct

Em Go, toda variável nasce com um "valor zero": `var m Money` é válido para o compilador e contém `0` centavos e moeda vazia. Não existe `null` para structs. Como o desafio exige rejeitar valores não inicializados, a moeda vazia é o marcador: `IsValid()` devolve falso, e toda operação e a serialização devolvem `ErrInvalidMoney`.

JSON

O pacote `encoding/json` procura dois métodos no tipo. Se existirem, usa-os no lugar do comportamento padrão. É o equivalente a um `JsonConverter` do C#.

```
func (m Money) MarshalJSON() ([]byte, error)      // Money  -> JSON
func (m *Money) UnmarshalJSON(data []byte) error  // JSON   -> Money
```

`UnmarshalJSON` é o único método com receiver de ponteiro (`*Money`), porque precisa preencher a variável de quem chamou. Se o conteúdo for inválido, a variável não é alterada.

O campo `amount` é declarado como `string`. Se o cliente mandar um número JSON (`"amount": 25.00`, sem aspas), a decodificação falha.

5. Decisões

DECISÃO	MOTIVO
`int64` em centavos, sem biblioteca decimal	Sem dependência externa; mapeia direto para `BIGINT` no Postgres
Limite: cerca de 92 quatrilhões por valor	É o teto do `int64` dividido por 100; fica documentado como limitação
Aceita `"25"`, `"25.0"` e `"25.00"`; todas viram `"25.00"`	São o mesmo valor, sem perda. A forma normalizada é a que entra no hash de idempotência (spec 07)
Rejeita `"25.000"`, mesmo sendo o mesmo valor	O desafio pede rejeição de escala excedente; aceitar zeros extras abriria exceção à regra
Rejeita zeros à esquerda, `+`, `-0.00`, `.50` e `25.`	Uma única forma de escrever cada valor reduz ambiguidade
Só moedas ISO 4217 de duas casas	A escala é fixa em 2; `JPY`, sem centavos, ficaria errado
Moeda em minúsculas é rejeitada	Não normalizar em silêncio
`ParseMoney` aceita negativo	Diferenças negativas são serializadas (reconciliação). Proibir negativo em aposta e saldo é regra das entidades, na spec 03

6. Conceitos de Go

CONCEITO	ONDE APARECE	PARALELO
Receiver por valor ou por ponteiro	`(m Money)` nos métodos; `(m *Money)` em `UnmarshalJSON`	Passagem por cópia ou por referência (`ref`)
Valor zero	`Money{}` é inválido de propósito	`default(T)` do C#
Erro sentinela + `errors.Is`	`ErrInvalidAmount` e os demais	`catch` por tipo de exceção
Múltiplos retornos	`Add` devolve `(Money, error)`	Tupla, ou padrão `TryParse`
Struct tags	`` `json:"amount"` ``	`[JsonPropertyName("amount")]`
Conjunto com `map[string]struct{}`	Lista de moedas aceitas	`HashSet<string>`; `struct{}` ocupa zero bytes
`t.Helper()`	`mustMoney` nos testes	Faz a falha apontar para a linha do teste, não para o helper

7. Como foi verificado

VERIFICAÇÃO	RESULTADO
`go test ./internal/domain/ -cover`	Passou; 100% das instruções
`go test -race ./...` (em container)	Passou
`gofmt -l .` e `go vet ./...`	Sem saída
Busca por `float32` e `float64` nos arquivos `.go`	Nenhuma ocorrência

O que os testes cobrem

- **Aceitos:** 12 formas válidas, incluindo os dois extremos do `int64`.
- **Rejeitados:** 23 entradas inválidas, de `NaN` a dígitos não ASCII.
- **Overflow:** no parsing, na soma, na subtração e na negação, incluindo os casos de fronteira que *não* devem estourar.
- **Moedas:** soma, subtração e comparação entre `BRL` e `USD` devolvem `ErrCurrencyMismatch`.
- **Não inicializado:** `Money{}` é rejeitado em sete operações.
- **JSON:** ida e volta, e oito corpos inválidos.

8. Perguntas de revisão

Por que `Add` devolve erro, se somar dois inteiros nunca falha em Go?

Por dois motivos: as moedas podem ser diferentes, e a soma pode passar do limite do `int64`. Go não avisa quando isso acontece; o valor dá a volta e fica negativo.

Por que `"25.000"` é rejeitado, se vale o mesmo que `"25.00"`?

A escala é fixa em duas casas e o desafio manda rejeitar escala excedente. Aceitar só quando os dígitos extras são zero seria uma exceção difícil de explicar no contrato.

Como o código garante que ninguém altera um `Money` depois de criado?

Os campos não são exportados e os métodos têm receiver por valor, então sempre devolvem um `Money` novo.

O que acontece com `var m Money` seguido de `m.Add(outro)`?

Devolve `ErrInvalidMoney`. O valor zero da struct tem moeda vazia, e isso é tratado como "não inicializado".

Por que `amount` é string no JSON, e não número?

Um número JSON seria lido como ponto flutuante por quase todo parser, inclusive no cliente. Como string, o valor chega exatamente como foi escrito.

Por que `ParseMoney` aceita `"-5.00"`, se o desafio proíbe valor negativo nas entradas?

`Money` também representa diferenças, que podem ser negativas. Quem proíbe negativo em uma aposta ou em um saldo é a entidade que recebe o valor, na spec 03.